

OpenDccCLib

Quick Start Guide — Decoder

Get a DCC decoder running in minutes

TI MSPM0G3507 LaunchPad + Code Composer Studio / Theia edition

OpenDccCLib supports any microcontroller with a C compiler — this guide covers one specific path.

Table of Contents

1. What is a DCC Decoder?
2. What You Need
3. Project Setup for Code Composer Studio
4. Understanding `dcc_user_config.h`
5. Building and Flashing
6. How the Decoder Works
7. Customizing Callbacks
8. What's Next

1. What is a DCC Decoder?

This guide uses a TI MSPM0G3507 LaunchPad with Code Composer Studio / Theia as a concrete, working example. OpenDccCLib itself is platform-agnostic C — it can be adapted to any microcontroller with a C compiler and GPIO input-capture. See the Developer Guide for other platforms and IDEs.

A DCC decoder is a device installed in a locomotive or accessory that listens to the DCC signal on the track rails, recognizes commands addressed to it, and acts on them — controlling motors, lights, servos, solenoids, or sound.

OpenDccCLib handles all the low-level work: classifying one/zero bits from edge timing, assembling packets, validating the XOR error check, matching addresses, and dispatching commands to your application callbacks. You write the code that drives your hardware.

Key Terms

- **Decoder:** The device that receives DCC commands and acts on them.
- **Command Station:** The device that generates DCC packets on the track.
- **Bit Decoder:** Classifies DCC one-bits (58µs half-period) from zero-bits (100µs+) based on edge timing.
- **Packet Decoder:** Assembles bits into bytes, validates XOR error check, and dispatches to callbacks.
- **Configuration Variable (CV):** A numbered setting stored in the decoder (address, speed curve, functions, etc.).
- **Service Mode:** Programming track mode where the command station reads/writes CVs. The decoder responds with a 6ms current pulse (ACK).
- **Failsafe:** If no valid DCC packet arrives for too long, the decoder enters failsafe mode (motor stop, outputs off).

2. What You Need

Hardware

- TI MSPM0G3507 LaunchPad development board (LP-MSPM0G3507)
- A DCC command station (commercial unit, OR a second LaunchPad running the command station demo)
- DCC signal input circuit (optocoupler or voltage divider to convert track-level DCC to 3.3V logic)
- USB cable for programming and UART terminal
- Optional: motor driver, LEDs, servos for output testing

DCC Signal Wiring

LaunchPad Pin	Function
PB1	Main track DCC signal input (edge-detect GPIO)
PB4	Service track DCC signal input (edge-detect GPIO)
PB17	Track select mux (LOW = main, HIGH = service)
PB3	Test pin (toggles on each decoded edge, for logic analyzer)
ACK pin	ACK current load output (6ms pulse for service mode)

Software

- TI Code Composer Studio (CCS) or CCS Theia IDE
- TI SysConfig tool (included with CCS)
- A serial terminal (PuTTY, Tera Term, or CCS Terminal) at 115200 baud
- The OpenDccCLib source code

3. Project Setup for Code Composer Studio

Starting From the Example

The library includes a ready-to-run decoder example at:

```
OpenDccCLib/applications/ti_theia/mspm03507_launchpad/decoder/
```

1. Open CCS/Theia and import the project (File > Import > CCS Projects).
2. Point it at the decoder folder.
3. The project references the DCC library source via a relative path to src/dcc/.

Complete Project File Structure

```
decoder/  
  decoder.c                <- Main entry point + edge ISR  
  dcc_user_config.h        <- Feature flags & limits  
  decoder_command_parser.c/h <- UART command interface  
  application_callbacks/  
    callbacks_dcc.c/h      <- Your motor/LED/servo logic  
  application_drivers/  
    ti_driverlib_dcc_driver.c/h <- Hardware abstraction (timers, GPIO)  
    ti_driverlib_uart_driver.c/h <- UART I/O  
    ack_pulse_driver.c/h    <- ACK current load control  
  dcc_lib/  
    dcc_config.h/c         <- Config struct & lifecycle API  
    dcc_bit_decoder.h/c    <- Edge timing classification  
    dcc_packet_decoder.h/c <- Packet assembly & dispatch  
    dcc_cv_storage.h/c     <- CV read/write with lock support  
    dcc_types.h            <- All data types  
    dcc_defines.h          <- NMRA protocol constants  
  Debug/  
    ti_msp_dl_config.c/h   <- SysConfig-generated init
```

4. Understanding dcc_user_config.h

For a decoder, the configuration file is simpler than a command station — you only need the decoder role and a function count.

```
// --- Role Selection ---
#define DCC_COMPILE_DECODER           // Bit decoder, packet parser, CV storage
// #define DCC_COMPILE_ACCESSORY_DECODER // Accessory-specific features

// --- Decoder Configuration ---
#define USER_DEFINED_DCC_DECODER_MAX_FUNCTIONS 29 // F0-F28
```

Do NOT define DCC_COMPILE_COMMAND_STATION in a decoder project. The decoder and command station roles can coexist in the same binary (for testing), but a real decoder only needs DCC_COMPILE_DECODER.

5. Building and Flashing

In Code Composer Studio

1. Open the imported decoder project.
2. Build the project (Project > Build or Ctrl+B).
3. Connect the LaunchPad via USB.
4. Flash and run (Run > Debug or F11).
5. Open a serial terminal at 115200 baud on the LaunchPad's COM port.
6. You should see: "DCC Decoder - MSPM0G3507 LaunchPad"

What Happens at Power-On

The `main()` function initializes hardware drivers and CV defaults (address 3 by default via CV1=3, CV29=0x06), passes the `dcc_config_t` struct to `DccConfig_initialize()`, then enters the main loop. The decoder immediately begins listening for DCC edges on PB1 (main track).

The Main Loop

```
while (1) {  
    _drain_edge_buffer();           // Feed buffered ISR timestamps to bit decoder  
    DccConfig_run();               // Library housekeeping & callback dispatch  
    CallbacksDcc_drain();          // Push RECV log lines to UART  
    TI_UartDriver_echo_process();  // Echo typed characters  
    DecoderCommandParser_process(); // Handle typed commands  
}
```

6. How the Decoder Works

Edge Capture (ISR)

The GPIO edge-detect ISR captures a microsecond timestamp into a 256-entry ring buffer. The ISR is minimal ($\sim 1\mu\text{s}$) to avoid overrunning the $58\mu\text{s}$ DCC half-bit period. No decoding happens in the ISR.

Bit Decoding (Main Loop)

The main loop drains the ring buffer and feeds each timestamp to `DccConfig_decoder_edge_isr()`. The bit decoder measures the interval between edges: $< 80\mu\text{s}$ = one-bit, $\geq 80\mu\text{s}$ = zero-bit (threshold set by `DCC_DECODER_HALF_BIT_THRESHOLD_US = 80`).

Packet Decoding

Once the bit decoder accumulates a valid preamble (10+ one-bits) followed by a start bit (zero), it collects bytes until the end bit (one). The packet decoder validates the XOR error check, matches the address against CV1/CV17-18/CV29, and dispatches to your application callbacks.

Address Matching

The library reads CV1 (short address), CV17-18 (long address), and CV29 (configuration register) during `DccConfig_initialize()`. CV29 bit 5 selects short vs. long addressing. The default demo uses short address 3.

7. Customizing Callbacks

All your application logic lives in `callbacks_dcc.c`. Each callback fires when the library decodes the corresponding DCC command. The demo implementation logs commands to UART; replace with your motor/LED/servo control code.

Speed Command Callback

```
void CallbacksDcc_on_speed_command(uint16_t address,
                                   uint8_t speed,
                                   bool direction,
                                   dcc_speed_mode_enum mode)
{
    // speed: 0 = stop, 1 = emergency stop, 2-126 = speed steps
    // direction: true = forward, false = reverse
    // mode: DCC_SPEED_MODE_14, _28, or _128
    set_motor_speed(speed, direction);
}
```

Function Command Callback

```
void CallbacksDcc_on_function_command(uint16_t address,
                                       uint8_t function_number,
                                       bool state)
{
    // function_number: 0 = headlight (FL), 1-68 = F1-F68
    // state: true = on, false = off
    if (function_number == 0)
        set_headlight(state);
}
```

All Available Callbacks

Callback	When It Fires
<code>on_speed_command</code>	Speed and direction command for this address
<code>on_emergency_stop_command</code>	Emergency stop for this address or broadcast
<code>on_function_command</code>	Function on/off (F0–F68)
<code>on_accessory_basic_command</code>	Basic accessory (turnout throw/close)
<code>on_accessory_extended_command</code>	Extended accessory (signal aspect)
<code>on_cv_write_command</code>	CV was written (notification after the fact)
<code>on_cv_verify_command</code>	CV verify command received
<code>on_cv_bit_command</code>	CV bit-level manipulation
<code>on_consist_command</code>	Consist (multi-unit) setup/clear
<code>on_failsafe_entered</code>	No valid DCC — stop motor, safe outputs
<code>on_failsafe_exited</code>	Valid DCC resumed

8. What's Next

Add RailCom Responses

Set `railcom_tx_pin_set` in `dcc_config_t` to a function that drives a GPIO connected to a current-source circuit. The library bit-bangs 4/8-encoded RailCom bytes at 250 kbaud during the cutout window. Also set `decoder_edge_irq_enable` to blank the edge interrupt during the cutout.

Add Persistent CV Storage

The demo stores CVs in a RAM array that resets on power cycle. Replace `CallbacksDcc_cv_read` and `CallbacksDcc_cv_write` with Flash or EEPROM read/write functions so CV changes survive power cycles.

Port to Another MCU

Copy the decoder project. Rewrite the driver files for your MCU's GPIO input-capture and timer API. The only ISR requirement is capturing a microsecond timestamp on each DCC signal edge.

Read the Developer Guide

The companion OpenDccCLib Decoder Developer Guide walks through every part of the system in detail: the config struct, bit decoder state machine, packet decoder, CV storage, address matching, failsafe logic, and more.